

Theoretical Recap — 14 May 2026 · Queues Theory

1. Queue Definition and Fundamental Properties

A **queue** is a linear abstract data type (ADT) that enforces **FIFO** (First-In, First-Out) ordering: the element that has been in the queue the longest is always the next one removed.

Linear nature

Elements form a single unbranched sequence; each element (except the front and rear) has exactly one predecessor and one successor. This sequential arrangement—shared by stacks and lists—classifies queues as linear structures, in contrast to non-linear structures such as trees and graphs.

Analogy

A queue behaves exactly like a line of people—new arrivals join at the back, service occurs at the front.

ADT vs Implementation

A queue ADT is a formal specification of behaviour. It defines a set of operations and the rules that govern them:

Operation	Behaviour
<code>enqueue(x)</code>	Add element <code>x</code> to the rear.
<code>dequeue()</code>	Remove and return the element at the front.
<code>peek()</code>	Return the front element without removal.
<code>isEmpty()</code>	Return <code>true</code> iff the queue contains zero elements.
<code>isFull()</code>	Return <code>true</code> iff the queue cannot accept more elements (relevant only for bounded implementations).

The ADT says nothing about memory layout, pointers, or indices. It is purely a logical contract.

Two standard implementation approaches exist:

1. **Array-based** – Uses a contiguous block of cells and tracks front and rear via indices. May be linear (wastes space) or circular (wraps indices with modulo arithmetic). Bounded by a fixed capacity unless a resizing strategy (e.g., doubling the array when full) is employed.
2. **Linked-list-based** – Uses a chain of nodes, each holding an element and a reference to the next node, with two external pointers: one to the front (head) and one to the rear (tail). Unbounded; `isFull()` always returns false.

The ADT is the contract; the implementation is the machinery. Conflating the two (e.g., writing “Queue = LinkedList”) is incorrect and loses marks. In Java, Queue is an interface (the ADT), and classes such as LinkedList or ArrayDeque provide concrete implementations. Analysis of efficiency and space utilisation always depends on which implementation is chosen.

1.4 When to Use a Queue — Typical Applications

Queues are the natural choice whenever **order of arrival must be preserved** and processing must happen in that same sequence. Common scenarios include:

- **CPU scheduling** (processes waiting for the processor)
- **Request buffering** (web server request queues, print spoolers)
- **Breadth-first search** (BFS) in graphs and trees
- **Asynchronous data transfer** (IO buffers, message queues)
- **Call centre systems** (holding lines for customers)

Any situation that can be modelled as “first-come, first-served” is a candidate for a queue.

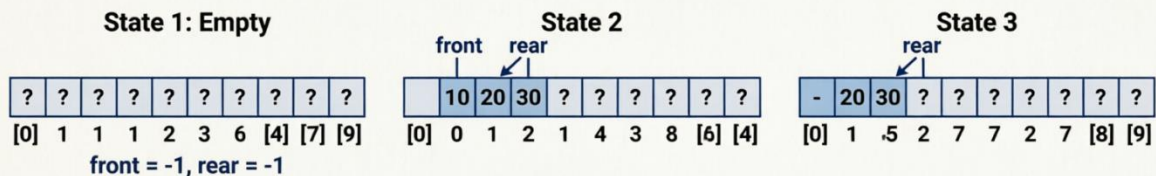
1.5 Advantages and Disadvantages of the Queue ADT

Advantages	Disadvantages
Simple, intuitive FIFO semantics — easy to reason about and implement.	Strict ordering — cannot prioritise or reorder elements without additional machinery (e.g., priority queue).
Fast operations — enqueue and dequeue are both $O(1)$ in well-designed implementations.	No random access — elements can only be accessed at the front, not by index.
Natural fit for many real-world problems — buffering, scheduling, breadth-first traversal.	Bounded implementations (array) require resizing or careful capacity management.
Clean separation of ADT and implementation allows choosing the best underlying structure (array vs linked list) for the context.	Linked-list implementations carry per-node pointer overhead and may suffer from cache-misses.

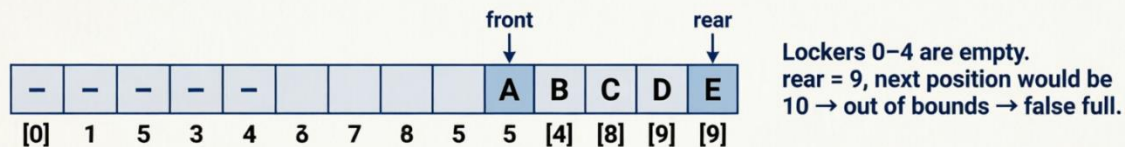
2. The Simple (Linear) Queue — The Row of Lockers and Its Wasted Space Problem

Computer Science: Queue Data Structure Fundamentals

1. Simple Linear Queue — Array as a Row of Lockers



2. The Wasted-Space Problem (Linear Queue)



2.1 The Array as a Row of Lockers

An array can be visualised as a straight row of lockers numbered from 0 to capacity-1. Two indices are maintained:

- `front` – the locker number of the first element in the queue.
- `rear` – the locker number of the last element in the queue.

2.2 Initial State and First Operations

When the queue is empty, no element occupies any locker, so both `front` and `rear` are set to -1.

- The first enqueue moves `rear` to 0, `front` to 0, and places the element in locker 0.
- A second enqueue moves `rear` to 1 while `front` remains at 0.
- A dequeue removes the element at `front` (locker 0) and moves `front` to 1, leaving locker 0 empty.

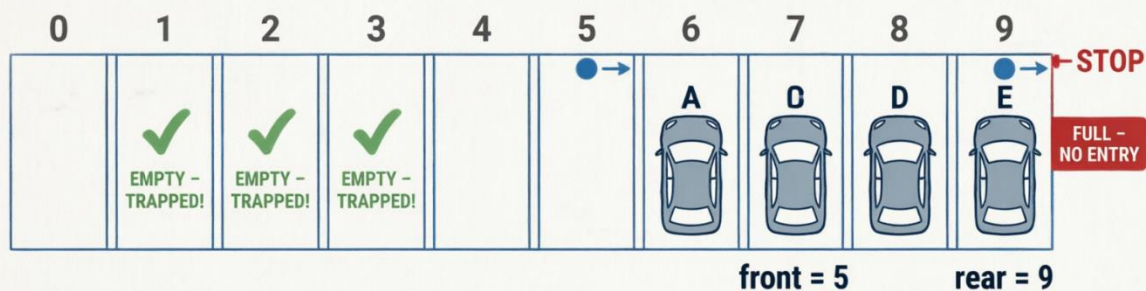
2.3 How the Problem Emerges

Assume capacity = 10. After many enqueues and dequeues, `front` may be at locker 5 and `rear` at locker 9. Lockers 5–9 are occupied; lockers 0–4 are empty (freed by earlier dequeues).

An enqueue now fails because rear is 9 and a linear queue simply increments rear to 10, which is out of bounds. The queue falsely reports full despite having five vacant cells at the front.

The root cause: rear only moves forward; it never wraps back. To reuse the front cells, all elements would need to be shifted downwards (from lockers 5–9 to 0–4), which is an $O(n)$ operation and defeats the purpose of fast $O(1)$ queue operations.

Linear Queue Wasted Space Problem: One-Way Parking Garage Analogy



Linear Queue Problem: Empty spots at front are permanently lost.
rear=9, next=10 is out of bounds = FALSE FULL despite 50% empty space.

Linear Queue = One-Way Parking Garage. Once cars leave from the front, those empty spots become permanently unreachable. The garage reports FULL even though half the spaces are empty – a false full condition.

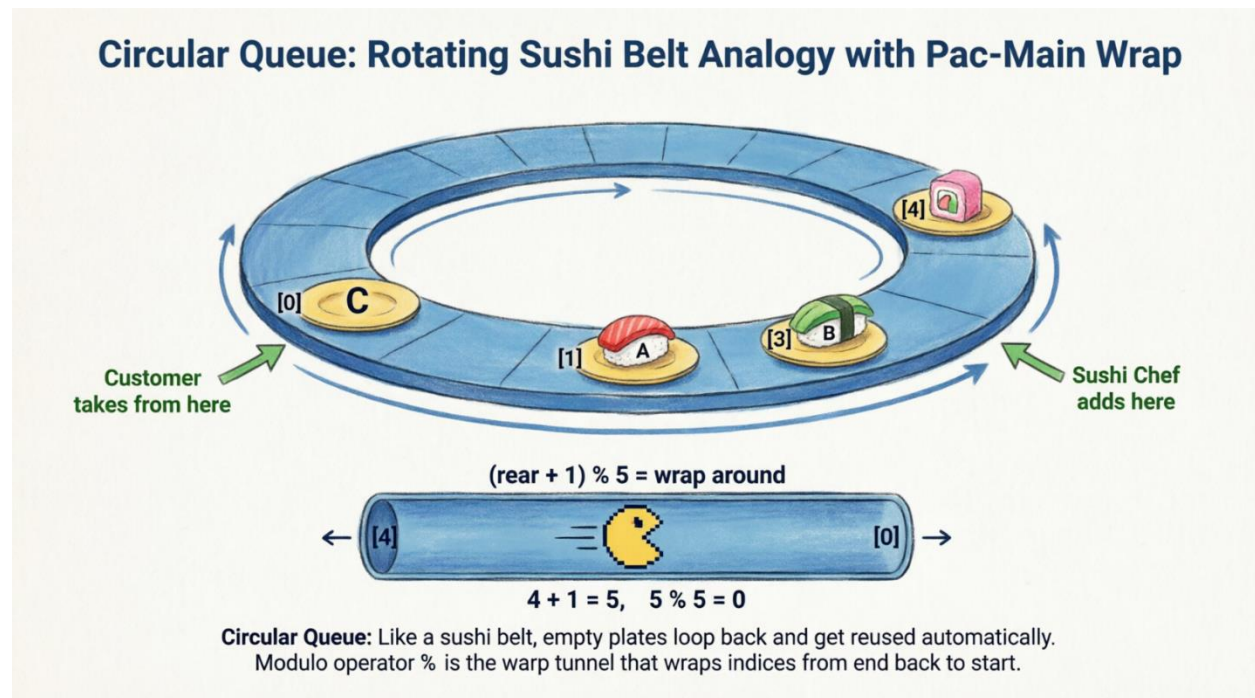
Key Insight: rear only moves forward; vacated front cells are lost forever unless we shift ($O(n)$).

2.4 Summary of the Flaw

- **Wasted space:** vacated front lockers are permanently lost.
- **False full condition.**
- **Shifting workaround:** $O(n)$, unacceptable for performance.

Thus, the circular queue is needed.

3. Circular Queue — The Circle of Lockers and Modulo Wrapping



1Circular Queue = Sushi Belt + Pac-Man Tunnel. Empty plates loop back automatically. The modulo operator % acts as the wrap tunnel – $(\text{rear} + 1) \% \text{capacity}$ sends you from the last locker back to locker 0.

No shifting. No wasted space. Just a wrap.

3.1 The Circle Analogy

If the lockers are arranged in a circle, after the last locker (e.g., 9) the next locker is 0. When rear reaches the last locker, the next enqueue simply wraps around to locker 0 if it is empty. The empty front slots are automatically reused with no shifting.

3.2 The Mathematical Tool: Modulo Operator

The modulo operator % provides the wrap-around arithmetic. For capacity n , the next position after index i is $(i + 1) \% n$.

Thus:

- New rear position after an enqueue: $(\text{rear} + 1) \% \text{capacity}$
- New front position after a dequeue: $(\text{front} + 1) \% \text{capacity}$

For example, with capacity 10, if $\text{rear} = 9$, the next position is 0; if $\text{rear} = 2$, the next is 3.

3.3 Detailed Step-by-Step Trace (capacity = 5, lockers 0–4) — with State Table

Below is a complete operation trace showing exactly how front and rear evolve, and how the array contents are reused. This goes further than a basic walkthrough to cement the wrap-around pattern.

Step	Operation	front	rear	Array (indices 0..4)	Notes
0	(empty)	-1	-1	[, , , ,]	Initial state
1	enqueue(A)	0	0	[A, , , ,]	First element
2	enqueue(B)	0	1	[A, B, , ,]	
3	enqueue(C)	0	2	[A, B, C, ,]	
4	enqueue(D)	0	3	[A, B, C, D,]	
5	dequeue() → A	1	3	[, B, C, D,]	front moves to 1, locker 0 empty
6	enqueue(E)	1	4	[, B, C, D, E]	rear reaches 4
7	dequeue() → B	2	4	[, , C, D, E]	
8	enqueue(F)	2	0	[F, , C, D, E]	rear wraps to 0 — locker 0 reused
9	enqueue(G)	2	1	[F, G, C, D, E]	queue full (5 elements)
10	dequeue() → C	3	1	[F, G, , D, E]	
11	dequeue() → D	4	1	[F, G, , , E]	
12	dequeue() → E	0	1	[F, G, , ,]	front wraps to 0
13	dequeue() → F	1	1	[, G, , ,]	only G remains
14	dequeue() → G	-1	-1	[, , , ,]	last element removed → empty

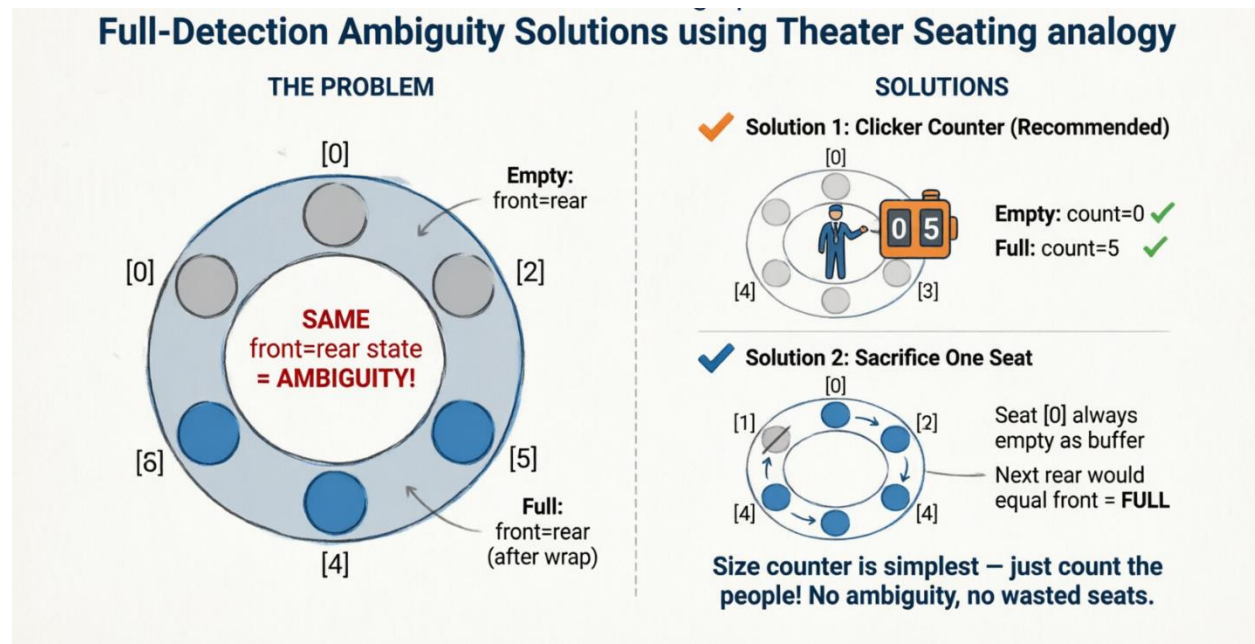
Circular Queue Walkthrough (capacity=5)

Step 1: Empty	[- - - - -]	
Step 2:	[A B C D -]	front=0 rear=3
Step 3:	[- B D E]	front=1 rear=4
Step 4:	[F - C D E]	front=2 rear=0
Step 5:	[F - - - -]	front=0
Step 6: Empty	[- - - - -]	

Observations from the trace:

- Whenever an index would go to 5, modulo sends it to 0 — the wrap.
- Empty cells are reused the moment they are vacated, never permanently abandoned.
- The queue can cycle through the array indefinitely without shifting any elements, as long as the number of elements does not exceed the capacity.

4. The Critical Full-Detection Problem — Why We Need an Explicit Rule



2 Empty and full can look identical ($\text{front} == \text{rear}$) — that's the ambiguity. Two clean solutions:

Clicker Counter (recommended) — just count the people. $\text{size} == 0 \rightarrow \text{empty}$, $\text{size} == \text{capacity} \rightarrow \text{full}$.

Sacrifice One Seat — leave one locker empty as a buffer. Full condition becomes $(\text{rear} + 1) \% \text{capacity} == \text{front}$.

Size counter = simplest, no wasted seats, no ambiguity.

4.1 The Ambiguity

Using only front and rear, an empty queue and a full queue can both satisfy the condition $(\text{rear} + 1) \% \text{capacity} == \text{front}$. Without additional information, the two states are indistinguishable.

4.2 Three Standard Solutions (With Exact Boolean Conditions)

Option 1: Maintain a size counter (recommended)

- An integer size tracks the current number of elements.
- The queue is empty when size equals 0.
- The queue is full when size equals capacity.

- No slot is sacrificed, and there is no ambiguity.

Option 2: Sacrifice one slot

- One cell is deliberately kept unused at all times, so the maximum number of elements is $\text{capacity} - 1$.
- The queue is empty when $\text{front} = -1$.
- The queue is full when the next position after rear equals front: $(\text{rear} + 1) \% \text{capacity} == \text{front}$.

Option 3: Boolean flag isFull

- A boolean flag is set to true whenever an enqueue causes the queue to reach its capacity, and cleared otherwise. This works but introduces extra state to maintain.

In any examination answer, the chosen strategy must be stated explicitly together with its Boolean condition. Option 1 is the cleanest and most intuitive.

5. Core Queue Operations (Circular Array, Size Counter)

The following descriptions assume an array of fixed capacity, with indices front and rear, and a counter size. They explain *what* each operation does and *how* it maintains the circular structure, without using code.

5.1 Enqueue

- **Purpose:** Insert an element at the rear.
- **Behaviour:**
 - If the queue is full (size equals capacity), the operation is rejected.
 - If the queue is empty (size equals 0), the front and rear are both set to index 0, the element is placed there, and size becomes 1.
 - Otherwise, the rear index is advanced circularly (using modulo arithmetic) and the element is stored at the new rear position; size increments.
- The front index remains unchanged unless the queue was empty.

5.2 Dequeue

- **Purpose:** Remove and return the front element.
- **Behaviour:**
 - If the queue is empty (size equals 0), the operation is rejected.
 - The element at the front index is retrieved.
 - size is decremented.
 - If the queue becomes empty (size reaches 0), both front and rear are reset to -1 to indicate the empty state.
 - Otherwise, the front index is advanced circularly.

5.3 Peek

- **Purpose:** Inspect the front element without removing it.
- **Behaviour:** If not empty, return the element at the front index; otherwise, the operation fails.

5.4 IsEmpty and IsFull

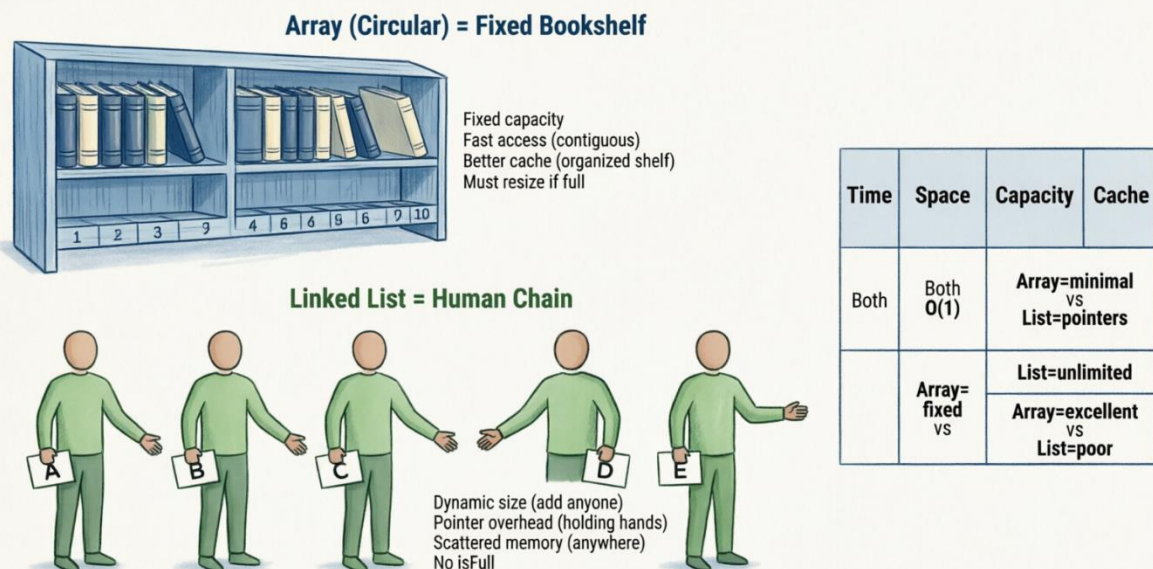
- **IsEmpty:** Returns true exactly when size equals 0.
- **IsFull:** Returns true exactly when size equals capacity.

5.5 Resize (When Capacity Is Exhausted)

- **Purpose:** Enlarge the array when the queue is full and further enqueues are required.
- **Behaviour:**
 - A new array of larger capacity is allocated (typically double the current capacity).
 - All existing elements are copied into the new array in the correct linear order, starting from the element currently at front and proceeding circularly through the old array for exactly size elements, placing them contiguously from index 0.
 - After copying, front is set to 0, rear to size - 1, and capacity is updated to the new size. The queue becomes linear in the new array with no wrapping until it fills again.

6. Linked-List Implementation — Theoretical

Queue Implementation Comparison: Fixed Bookshelf vs Human Chain



A queue can also be realised with a singly linked list, using two external references: a **head** pointer to the front node and a **tail** pointer to the rear node.

6.1 Structural Overview

- Each node holds an element (the data) and a reference to the next node.
- The head pointer identifies the first element (front); the tail pointer identifies the last element (rear).
- A size counter may be maintained but is not strictly required; the queue is empty when the head pointer is null.

6.2 Operation Theory

- **Enqueue:** A new node is created for the incoming element.
 - If the queue is empty (head is null), both head and tail are set to this new node.
 - Otherwise, the current tail node's next reference is linked to the new node, and tail is updated to point to it.
This adds the element at the rear in constant time, independent of the number of elements.
- **Dequeue:**
 - If the queue is empty (head is null), the operation is rejected.
 - The element stored in the head node is retrieved.
 - Head is advanced to the next node in the chain.
 - If head becomes null (the queue is now empty), tail is also set to null.
 - The old front node becomes unreachable and is freed.
Removal from the front is $O(1)$.
- **Peek:** Return the element in the head node, if the queue is not empty; $O(1)$.
- **IsEmpty:** True when head is null (or size equals 0). $O(1)$.
- **IsFull:** Always returns false for a linked list, as no fixed capacity bounds the structure (unless a size limit is artificially imposed). Memory exhaustion is a separate concern.

6.3 Trade-offs Between Array and Linked-List Implementations

Aspect	Array (Circular)	Linked List
Enqueue / Dequeue time	$O(1)$	$O(1)$
Memory overhead	Minimal, contiguous allocation	Per-node overhead for pointers
Cache performance	Excellent (contiguous)	Poorer (non-contiguous nodes)
Capacity	Fixed or requires resizing	Dynamically grows, no <code>isFull</code>
Implementation complexity	Must handle modulo and full detection	Simple pointer manipulation

Both provide $O(1)$ enqueue and dequeue; the choice depends on constraints such as bounded capacity needs, memory overhead tolerance, and cache behaviour requirements.

7. Efficiency Comparison — Why Circular Queue Wins Over Linear Array Queue

Implementation	Enqueue	Dequeue	Space utilisation
Linear queue with shifting	$O(1)$	$O(n)$	Full, but time penalty
Linear queue, pointer-only, no shift	$O(1)$	$O(1)$	Vacated front cells permanently lost
Circular queue (modulo)	$O(1)$	$O(1)$	All cells reused — fully efficient

Key insight: The circular queue’s advantage over the pointer-only linear queue is **space utilisation**, not speed. All core operations on a circular queue are $\Theta(1)$ in best, average, and worst cases.

8. Common Mistakes and Misinterpretations

Mistake	Why It’s Wrong	Correct View
“Queue = LinkedList”	Confuses the ADT with one concrete class.	Queue is a FIFO behaviour contract; it can be implemented with an array or a linked list.
Clockwise vs anti-clockwise	Visual direction is irrelevant.	The wrap is fully defined by $(\text{index} + 1) \% \text{capacity}$.
Unanswered “How do we know the array is full?”	Leaves a testable condition unspecified.	Must give an explicit Boolean expression (size counter or sacrificed slot).
BST, Heap, Graph listed as linear	These have branching or networked connections.	Linear: Array, LinkedList, Stack, Queue. Non-linear: BST, Heap, Graph.

10. Memory Hooks

- **Queue** — linear ADT, FIFO: enqueue at rear, dequeue from front.
- **Circular Queue** — array-based queue where indices wrap via $(\text{index} + 1) \% \text{capacity}$, reusing vacated cells without shifting.
- **Linked-List Queue** — dynamic queue using head/tail references; all operations $O(1)$, unbounded capacity.
- **Stack** — linear ADT, LIFO: push and pop at the top.
- **ADT** — specification of *what* operations a data structure supports, independent of *how* it is built.

11. Coverage Index: What, Why, How, When, Advantages/Disadvantages, Trade-offs

Aspect	Where Covered
What	§1 (ADT definition, FIFO, linear nature, analogy), §2.1–2.2 (linear array representation)
Why	§2.3–2.4 (why linear queue fails), §3 (why circular queue solves it), §4 (why full-detection is needed)
How	§3.2–3.3 and the trace table (modulo wrapping, pointer updates), §5 (enqueue, dequeue, peek, resize behaviour), §6 (linked-list operations)
When	§1.4 (typical applications), §6.3 trade-offs table (when to choose array vs linked list), §7 (when circular queue preferred over linear)
Advantages / Disadvantages	§1.5 (ADT-level pros/cons), §4.2 (pros/cons of full-detection strategies), §6.3 (trade-offs between implementations), §7 (advantage of circular over linear queue)
Trade-offs	§4.2 (size counter vs sacrificed slot vs flag), §6.3 (array vs linked list), §7 (linear shifting vs pointer-only vs circular)
Common mistakes	§8 (misconceptions and corrections)